

AI Usage Report

Course: SMD

Assignment: 1

Student Name: Humair Naseer

Registration No.: 22i-2632

Date: 17 Sep 2026

1. AI Tool(s) Used

List the AI tools used during the development of your application.

Response:

I used Claude AI (Anthropic) as my primary AI assistant throughout the development of this React Native application. Claude was accessed through the Kiro VS Code extension, which provided an integrated development environment for AI-assisted coding.

2. Purpose of AI Usage

Explain what you used AI for during the development of your application.

Response:

I used AI assistance for the following purposes:

- 1. Architecture Planning:** Designing the overall application structure, including component organization, data flow, and screen navigation patterns using state-based view switching
- 2. React Native Concepts:** Understanding and implementing React hooks (useState), component lifecycle, props passing, and conditional rendering patterns
- 3. Component Development:** Creating reusable UI components (Button, Card, Input, Header) with proper prop handling and style management
- 4. Chart Implementation:** Implementing react-native-chart-kit for data visualization, understanding chart configuration, and creating meaningful dashboard analytics
- 5. Form Validation:** Implementing form input validation with regex patterns for email validation and real-time error feedback
- 6. Data Structure Design:** Creating comprehensive mock data structures with nested objects and arrays for courses, student information, and academic records
- 7. JavaScript Logic:** Implementing array methods (.map, .filter, .reduce, .sort) for data manipulation, search, and sorting functionality
- 8. State Management:** Understanding state-based navigation, managing multiple component states, and handling state updates

9. **Code Optimization:** Refactoring code for better reusability, removing unnecessary dependencies, and improving code organization
10. **Debugging Assistance:** Identifying and fixing issues with component rendering, navigation flow, and data binding
-

3. Important Prompts Used

Provide the important prompts that you gave to the AI tool.

Prompt 1:

"Create a React Native student portal application with the following requirements: state-based navigation (no navigation libraries), dashboard with at least 4 chart types using react-native-chart-kit, attendance monitoring with smart warnings, course management with search and sort functionality, form validation, and data-driven UI using mock data. The app should demonstrate React concepts including components, props, state, events, conditional rendering, and reusable components. Include meaningful interactions and avoid static screens."

Prompt 2:

"Implement a Profile screen with an editable form that includes: name and email input fields with validation (name minimum 3 characters, email format validation using regex), real-time error display, loading state during submission, and success feedback. Use React Native's built-in TextInput component properties for validation. The form should demonstrate state management, controlled inputs, and proper error handling."

Prompt 3:

"Fix the navigation implementation - the assignment requires state-based view switching as taught in class, not React Navigation library. Replace the NavigationContainer and Stack.Navigator with a simple useState approach where currentScreen state determines which screen to render using a switch statement. This avoids negative marking for using external navigation code."

Prompt 4:

"Create a comprehensive dashboard with analytics tab that displays 4 different chart types from react-native-chart-kit: Line Chart for CGPA trend over semesters, Bar Chart for course-wise attendance comparison, Pie Chart for credit hours distribution, and Progress Chart for overall academic status. Each chart should display meaningful data and include insights below explaining what the data shows."

Prompt 5:

"Implement interactive search and sort functionality in the Courses screen. Search should filter courses in real-time by course name, code, or instructor name using .filter() method. Include three sort options (by name, attendance, credits) with visual feedback showing active sort. When no results found, display an EmptyState component with appropriate message and action button."

Prompt 6:

"Create reusable components following React best practices: Button component with variants (primary, secondary, success, danger) and sizes (small, medium, large), Card component with optional onPress for tap functionality, Input component with validation error display, and Header component with back navigation and customizable right content. Each component should accept props and use StyleSheet for styling."

4. AI-Generated Output

Briefly describe the code, solution, or suggestion provided by the AI.

Response:

The AI provided:

1. **Complete Project Structure:** Generated organized folder structure with separate directories for components/, screens/, and data/, following React Native best practices
 2. **State-Based Navigation System:** Created App.js with useState for screen management and switch/case rendering pattern, replacing external navigation libraries
 3. **Reusable Component Library:** Generated 7 reusable components (Button, Card, Input, Header, CourseCard, StatCard, EmptyState) with proper prop handling and consistent styling
 4. **Dashboard with Charts:** Implemented complete DashboardScreen with tab navigation, 4 chart types (Line, Bar, Pie, Progress), and data transformation logic for visualization
 5. **Mock Data Layer:** Created comprehensive mockData.js with student information, 5 courses with nested assignments/quizzes, announcements, fee details, semester grades, and helper functions for calculations
 6. **Form Validation Logic:** Generated ProfileScreen with controlled inputs, validation functions (email regex, required fields, minimum length), error state management, and success/error feedback
 7. **Interactive Features:** Implemented search functionality using .filter(), sort options with .sort(), pull-to-refresh, tab switching, and navigation between screens
 8. **Calculations and Logic:** Created helper functions for attendance percentage calculation, course marks totaling, GPA calculations, and "classes needed" formula
 9. **Conditional Rendering:** Implemented multiple application states including loading states, empty states (no search results), error states (validation), and data-dependent states (attendance color coding)
 10. **Comprehensive Documentation:** Generated multiple markdown files explaining the project structure, requirements coverage, testing guide, and troubleshooting steps
-

5. Changes Made by Me

Explain what you changed, modified, or improved in the AI-generated solution.

Response:

I made the following modifications and improvements:

1. **Navigation Correction:** AI initially generated code using React Navigation library. I identified this violated assignment requirements and requested correction to state-based navigation. Verified the change was implemented correctly and removed unnecessary dependencies from package.json.
2. **Data Customization:** Modified mockData.js to include more realistic university context - changed student information to match our university structure, adjusted course codes and names to match typical CS curriculum, and added relevant Pakistani fee amounts.

3. **Validation Rules:** Enhanced form validation by adding phone number format validation for Pakistani numbers (+92 format) and extended the minimum name length requirement based on testing with various inputs.
 4. **UI Polish:** Adjusted color scheme to be more visually appealing, increased touch target sizes for better mobile usability, and refined spacing and padding values for better visual hierarchy.
 5. **Chart Insights:** Added personalized insight text below each chart explaining what the data represents and what students should learn from it, making the dashboard more educational.
 6. **Empty State Messages:** Customized empty state messages to be more helpful and contextual - for example, when no courses found in search, the message now suggests alternative search terms.
 7. **Testing and Debugging:** Tested the app extensively on mobile device, identified and fixed issues with chart rendering on first load, resolved state update issues in search functionality, and ensured all navigation paths work correctly.
 8. **Code Comments:** Added explanatory comments to complex logic sections, particularly in calculation functions and validation logic, to ensure code maintainability.
 9. **Performance Optimization:** Identified unnecessary re-renders in some components and optimized by properly structuring state updates and conditional rendering logic.
 10. **Documentation Review:** Reviewed all generated documentation files, removed emoji usage as requested, and ensured all explanations accurately reflect the implemented features.
-

6. My Understanding

Explain the important part(s) of the AI-generated code in your own words.

Response:

State-Based Navigation:

The navigation works by maintaining a `currentScreen` state variable using `useState` hook. When user taps a button or card, we call `setCurrentScreen()` with the new screen name, which triggers React to re-render. The `renderScreen()` function uses a switch statement to check `currentScreen` value and returns the appropriate screen component. This is simpler than navigation libraries and demonstrates understanding of React state management.

Form Validation:

The validation system uses controlled inputs where form values are stored in state. Each input field has an `onChange` handler that updates state. When form is submitted, `validateProfileForm()` function checks each field - it uses regex pattern `/^[^\s@]+@[^\s@]+\.[^\s@]+$/` for email format, checks string length for minimum characters, and uses `trim()` to handle whitespace. Errors are stored in an `errors` object and displayed conditionally below each input field using `error prop`.

Data-Driven UI:

Instead of hardcoding UI elements, the interface is generated from data arrays. For example, `courses.map()` creates a `CourseCard` for each course object. This means if we add a new course to the `courses` array in `mockData.js`, it automatically appears in the UI without changing screen code. The same pattern is used for announcements, assignments, and semester grades.

Array Methods for Search:

The search works using `.filter()` method which creates a new array containing only courses that match the search query. The condition `course.name.toLowerCase().includes(searchQuery.toLowerCase())` makes it case-insensitive. For sorting, `.sort()` compares two courses based on selected criteria (name uses `localeCompare` for alphabetical order, attendance calculates percentage and compares numbers).

Chart Implementation:

Each chart requires a data object with specific structure. For Line Chart, we need labels array (semester names) and datasets array containing data points (GPA values). The `map()` method transforms `semesterGrades` array into the required format. `chartConfig` object controls appearance including colors, fonts, and decimal places. The charts are responsive because we use `Dimensions.get('window').width` to calculate chart width.

Conditional Rendering:

The app uses multiple conditional rendering patterns. Ternary operator `{condition ? trueOutput : falseOutput}` is used for simple conditions like showing different badges. For complex conditions, we use `if` statements or separate render functions. Tab switching uses `{selectedTab === 'overview' && renderOverview()}` pattern. Empty states appear when `array.length === 0`.

Reusable Components:

Components like `Button` accept props (title, onPress, variant, size) which customize behavior and appearance. The component uses array syntax `[styles.button, styles[variant], styles[size]]` to combine multiple styles. This makes one `Button` component reusable in many places with different looks. Props are destructured in function parameters for cleaner code.

Calculations:

Attendance percentage uses formula $(\text{attended} / \text{total}) \times 100$ with `.toFixed(1)` for one decimal place. The "classes needed" calculation solves the equation $(\text{attended} + x) / (\text{total} + x) = 0.75$ for x , which gives $x = (0.75 \times \text{total} - \text{attended}) / 0.25$. `Math.ceil()` rounds up because you can't attend partial classes. These formulas are encapsulated in helper functions for reusability.

7. Verification and Testing

Explain how you tested the AI-generated solution.

Response:

Initial Testing:

After AI generated the code, I first ran ``npm install`` to install dependencies, then ``npm start`` to launch the app. The app loaded on my Android phone via Expo Go successfully. I systematically tested each screen by navigating through the app.

Issues Encountered:

1. **Chart Rendering Issue:** On first load, charts in Analytics tab showed blank. This was because chart data transformation wasn't completing before render. I fixed this by ensuring data calculations happened before chart components mounted.

2. **Navigation Library Problem:** Initially used React Navigation which violated assignment requirements. Replaced with state-based navigation using useState and switch statement. Tested all navigation paths to ensure back buttons and screen transitions work correctly.

3. **Search Not Updating:** Search input was typing but course list not filtering. Discovered the filtered array wasn't being used in the map() function. Fixed by ensuring sortedCourses array uses filteredCourses as base.

4. **Form Validation Timing:** Validation errors were showing immediately on load before user interaction. Modified to only show errors after field is touched or form is submitted.

5. **Empty State Logic:** "No courses found" appeared even when courses existed. Fixed the condition from checking courses.length to checking filteredCourses.length.

Testing Methodology:

1. **Functional Testing:** Tested every button, input field, and interactive element to ensure it responds correctly. Verified all navigation paths work and back buttons return to correct screens.

2. **Data Testing:** Modified mockData.js to test different scenarios - changed attendance to below 75% to verify warnings appear, set announcements array to empty to test empty state, added new course to verify data-driven UI updates.

3. **Validation Testing:** Tested form with invalid inputs - empty name, short name, invalid email formats (missing @, missing domain, etc.). Verified appropriate error messages display and prevent submission.

4. **State Testing:** Tested all application states - loading state when saving profile, empty state when search returns no results, error state with invalid form data, success state after form submission.

5. **Calculation Verification:** Manually calculated expected values and compared with app output. For example, Database Systems with 22/30 attendance should show 73.33% and warning. Verified "attend next 5 classes" calculation is correct.

6. **Chart Testing:** Verified all 4 charts display with correct data. Line chart shows 6 semesters with proper GPA values, bar chart displays all 5 courses with attendance percentages, pie chart shows correct credit distribution, progress chart displays three metrics correctly.

7. **Device Testing:** Tested on actual Android device (not just emulator) to verify touch targets are appropriately sized, text is readable, scrolling is smooth, and overall usability is good on mobile screen.

Verification Results:

After fixes, all features work correctly. Navigation flows smoothly between all 6 screens, search and sort functions filter correctly, form validation catches all error cases, charts display all data properly, and calculations match expected values. No console errors or crashes during normal usage.

8. Reflection

What did you learn from using AI during this assignment?

Response:

Using AI for this assignment was highly educational in several ways:

Technical Learning:

I gained deep understanding of React Native concepts that were previously unclear. Seeing AI generate complete components helped me understand the pattern of props destructuring, state management with `useState`, and event handling. I learned how to structure a React Native project properly with organized folders and reusable components. The chart implementation taught me how to work with third-party libraries and transform data into required formats.

Problem-Solving Skills:

When AI-generated code had issues (like using navigation library), I had to identify the problem, understand why it violated requirements, and request appropriate corrections. This improved my debugging skills and ability to critically evaluate code against requirements. I learned that AI outputs need verification and testing, not blind acceptance.

Code Understanding:

By modifying and debugging AI-generated code, I was forced to understand how every part works. Reading through the validation logic, I learned regex patterns for email validation. Examining the search implementation taught me proper use of `.filter()` and `.includes()` methods. Studying the chart configuration helped me understand how data visualization works in React Native.

Best Practices:

I learned professional code organization patterns - separating data from UI, creating reusable components, writing helper functions for calculations, and using meaningful variable names. The AI followed React best practices which I can now apply in future projects.

Limitations of AI:

I discovered AI doesn't always understand specific assignment constraints (like "no navigation libraries"). I had to clearly specify requirements and verify outputs match assignment guidelines. This taught me the importance of detailed prompts and thorough review of generated code.

Practical Development:

I learned the complete development workflow - setting up project, installing dependencies, running development server, testing on device, debugging errors, and deploying. The documentation AI generated taught me the importance of clear code documentation and README files for project understanding.

Independent Learning:

When fixing issues AI couldn't solve alone, I had to search React Native documentation and understand concepts independently. This reinforced that AI is a learning assistant, not a replacement for understanding fundamentals.

Overall Impact:

AI accelerated my learning curve significantly. Instead of spending hours searching for syntax or basic patterns, I could focus on understanding concepts and building features. However, I recognize that simply copying AI code wouldn't help me learn - the key was actively modifying, testing, debugging, and understanding every piece of generated code. This assignment taught me how to effectively use AI as a development tool while maintaining deep understanding of the code I submit.

Student Declaration

I confirm that I have used AI tools only as a development assistant and that I understand the code submitted as part of this assignment. I am able to explain and demonstrate the functionality of my application.

Student Name: Humair Naseer

Signature: Humair Naseer

Date: 17 Sep 2026